

AW87XXX Android Driver(MTK)

版本： V3. 6

时间： 2023 年 12 月

目录

1. 驱动说明	3
2. 驱动移植	3
2.1 AW87XXX 驱动移植	3
2.1.1 DTS 配置	3
2.1.2 驱动配置	4
2.1.3 BIN 文件配置	5
2.2 平台驱动配置	6
2.2.1 KCONTROL 注册	6
2.2.2 WIDGET 配置	6
2.3 低电量保护功能	8
2.3.1 DSP 通信接口配置	8
2.4 驱动移植有效性验证	8
3. 调试接口	10
3.1 设备节点	10
4. 附录	12
4.1 常见问题	12
4.1.1 移植 AWINIC 算法后出现白噪	12

1. 驱动说明

驱动源码文件	aw87xxx.c, aw87xxx.h, aw87xxx_device.c, aw87xxx_device.h, aw87xxx_monitor.c, aw87xxx_monitor.h, aw87xxx_dsp.c, aw87xxx_dsp.h, aw87xxx_acf_bin.c, aw87xxx_acf_bin.h, aw87xxx_log.h, aw87xxx_bin_parse.c, aw87xxx_pid_39_reg.h, aw87xxx_pid_59_3x9_reg.h, aw87xxx_pid_59_5x9_reg.h, aw87xxx_pid_5a_reg.h, aw87xxx_pid_18_reg.h, aw87xxx_pid_9b_reg.h, aw87xxx_pid_76_reg.h, aw87xxx_pid_60_reg.h, aw87xxx_pid_c1_reg.h, aw87xxx_pid_c2_reg.h
驱动支持产品	aw87319, aw87329, aw87339, aw87349, aw87359, aw87389, aw87509, aw87519, aw87529, aw87539, aw87549, aw87559, aw87569, aw87579, aw81509, aw87579G aw87390, aw87360, aw87401, aw87390G aw87358, aw87560, aw87550 aw87391, aw87392 aw87565, aw87566, aw81564, aw87567, aw87568
I ² C 地址范围	aw87358:0x5C 其他产品:0x58, 0x59, 0x5A, 0x5B

2. 驱动移植

2.1 AW87XXX 驱动移植

2.1.1 DTS 配置

AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G/AW87391/AW87392 产品没有复位引脚，不能配置 **reset-gpio**。

产品表信息表

产品	产品 ChipID	是否有 reset-pin
AW87319	0x9B	是
AW87329/AW87339/AW87349	0x39	是
AW87519/AW87529	0x59	是
AW87359/AW87389	0x59	否
AW87549	0x5A	否
AW87559/AW87569/AW87579/AW81509/AW87579G	0x5A	是
AW87390/AW87360/AW87401/AW87390G	0x76	否
AW87358	0x18	是
AW87560/AW87550	0x60	是
AW87391/AW87392	0xC1	否
AW87565/AW87566/AW81564/AW87567/AW87568	0xC2	是

单 PA 配置方法

```
&i2c_x {                                     /*x 表示 I2C 总线*/
    aw87xxx_pa@58 {                          /*以 I2C 地址 0x58 举例*/
        compatible = "awinic,aw87xxx_pa";
        reg = <0x58>;
        /*无 reset-pin 的产品不能配置 reset-gpio，否则无法正常加载配置*/
        reset-gpio = <&pio 106 0>; /*reset-gpio 以 gpio 106 举例*/
        dev_index = < 0 >;                /*单 PA dev_index 为 0*/
        status = "okay";
    };
};
```

多 PA 配置方法

```
&i2c_x {                                     /*x 表示 i2c 的总线号*/
    aw87xxx_pa@58 {
        compatible = "awinic,aw87xxx_pa";
        reg = <0x58>;
        /*无 reset-pin 的产品不能配置 reset-gpio，否则无法正常加载配置*/
        reset-gpio = <&pio 106 0>;
        dev_index = < 0 >;                /*第 1 个 PA dev_index 为 0*/
        status = "okay";
    };
    aw87xxx_pa@59 {
        compatible = "awinic,aw87xxx_pa";
        reg = <0x59>;
        /*无 reset-pin 的产品不能配置 reset-gpio，否则无法正常加载配置*/
        reset-gpio = <&pio 107 0>;
        dev_index = < 1 >;                /*第 2 个 PA dev_index 为 1*/
        status = "okay";
    };
};
/*以上以双 PA 举例，多 PA 时 dev_index 依次增加，其他属性参考单 PA 配置*/
```

2.1.2 驱动配置

defconfig 编译配置选项:

```
CONFIG_SND_SOC_AW87XXX=y
```

内核 codecs 目录下创建 aw87xxx 目录，添加驱动文件:

```
aw87xxx.c, aw87xxx_device.c, aw87xxx_monitor.c, aw87xxx_dsp.c, aw87xxx_acf_bin.c,
aw87xxx_bin_parse.c aw87xxx.h, aw87xxx_device.h, aw87xxx_monitor.h, aw87xxx_dsp.h,
aw87xxx_acf_bin.h, aw87xxx_log.h, aw87xxx_pid_39_reg.h, aw87xxx_pid_59_3x9_reg.h,
aw87xxx_pid_59_5x9_reg.h, aw87xxx_pid_5a_reg.h, aw87xxx_pid_18_reg.h,
```

aw87xxx_pid_9b_reg.h, aw87xxx_pid_76_reg.h, aw87xxx_pid_60_reg.h, aw87xxx_pid_c1_reg.h,
aw87xxx_pid_c2_reg.h

Kconfig 配置:

```
config SND_SOC_AW87XXX
    tristate "SoC Audio for awinic AW87XXX Smart K PA"
    depends on I2C
    help
        This option enables support for AW87XXX Smart K PA.
```

Makefile 配置:

```
obj-$(CONFIG_SND_SOC_AW87XXX) += aw87xxx/aw87xxx.o
aw87xxx/aw87xxx_device.o aw87xxx/aw87xxx_monitor.o
aw87xxx/aw87xxx_bin_parse.o aw87xxx/aw87xxx_dsp.o
aw87xxx/aw87xxx_acf_bin.o
```

2.1.3 bin 文件配置

PA 需要配置寄存器等参数才能正常工作, PA bin 文件配置步骤如下:

编译配置

在项目对应位置添加 bin 文件编译选项:

```
PRODUCT_COPY_FILES += \
    xxxx/aw87xxx_acf.bin:$(TARGET_COPY_OUT_VENDOR)/firmware/aw87xxx_acf.bin

/*xxxx 为平台路径*/
```

路径配置

在内核 firmware_class.c 文件中添加 bin 文件在手机中的目录, 一般目录为/vendor/firmware

```
static const char * const fw_path[] = {
    fw_path_para,
    "/vendor/firmware", /*添加路径*/
    "/lib/firmware/updates/" UTS_RELEASE,
    "/lib/firmware/updates",
    "/lib/firmware/" UTS_RELEASE,
    "/lib/firmware"
};
```

/* 注: 调试阶段可直接 push bin 文件到 /vendor/firmware/ */

bin 文件选择

以 aw87390 为例, bin 文件包含 Music/Receiver/Off 场景:

```

aw87xxx_driver                                /*驱动根目录*/
├── config                                    /*PA配置路径*/
│   ├── aw87390
│   │   ├── mono
│   │   │   └── aw87xxx_acf.bin                /*单PA 配置*/
│   │   └── stereo
│   │       └── aw87xxx_acf.bin                /*双PA 配置*/
│   └── monitor_bin_guide                    /*monitor bin目录*/
│       ├── aw87xxx_monitor.bin
│       ├── aw87xxx_monitor_bin_guide.pdf
│       ├── monitor.exe
│       └── monitor_params.txt

```

2.2 平台驱动配置

2.2.1 kcontrol 注册

aw87xxx_codec 注册:

```

--- a/mtk-soc-codec-6357.c
+++ b/mtk-soc-codec-6357.c
/*以 mt6357平台为例*/

/*添加接口声明*/
#ifdef CONFIG_SND_SOC_AW87XXX
+ extern int aw87xxx_add_codec_controls(void *codec);
#endif

/*添加调用注册*/
@@ -5361,6 +5361,15 @@ static int mt6357_codec_probe(struct snd_soc_codec
*codec)
+
+     ARRAY_SIZE(mt6357_pmic_test_controls));
+     snd_soc_add_codec_controls(codec, Audio_snd_auxadc_controls,
+     ARRAY_SIZE(Audio_snd_auxadc_controls));
#ifdef CONFIG_SND_SOC_AW87XXX
+     err = aw87xxx_add_codec_controls((void *)codec);
+     if (err < 0) {
+         pr_err("%s: add_codec_controls failed, err %d\n",
+         __func__, err);
+         return err;
+     };
#endif
/* here to set private data */
mCodec_data = kzalloc(sizeof(struct mt6357_codec_priv), GFP_KERNEL);
if (!mCodec_data) {

```

2.2.2 widget 配置

MTK 4G 平台与 5G 平台配置有差异, 参考如下:

4G 平台配置方法

以单 PA 举例：

```
/*以 mt6357 平台为例*/
/*添加驱动接口及场景定义*/
@@ -80,6 +80,17 @@ static void setDlMtkifSrc(bool enable);
#ifdef ANALOG_HPTRIM
static int SetDcCompenSation(bool enable);
#endif
+#ifdef CONFIG_SND_SOC_AW87XXX
+extern int aw87xxx_set_profile(int dev_index, char *profile);
+
+static char *aw_profile[] = {"Music", "Off"}; /*aw87xxx_acf.bin 文件中配置场景*/
+enum aw87xxx_dev_index {
+    AW_DEV_0 = 0,
+};
+#endif
static void Voice_Amp_Change(bool enable);

/* 添加场景设置函数 */
@@ -3296,6 +3347,9 @@ static void Speaker_Amp_Change(bool enable)
    Ana_Set_Reg(AUDDEC_ANA_CON6, 0x0201, 0xffff);
    /* Switch LOL MUX to audio DAC */
    Ana_Set_Reg(AUDDEC_ANA_CON4, 0x011b, 0xffff);
+#ifdef CONFIG_SND_SOC_AW87XXX
+    /*切换 PA AW_DEV_0 为 Music 场景*/
+    aw87xxx_set_profile(AW_DEV_0, aw_profile[0]);
+#endif
    /* disable Pull-down HPL/R to AVSS28_AUD */
    if (mIsNeedPullDown)
        hp_pull_down(false);
@@ -3333,6 +3387,10 @@ static void Speaker_Amp_Change(bool enable)
    Ana_Set_Reg(AUDDEC_ANA_CON12, 0x0, 0x1055);
    /* Disable NCP */
    Ana_Set_Reg(AUDNCP_CLKDIV_CON3, 0x1, 0x1);
+
+#ifdef CONFIG_SND_SOC_AW87XXX
+    /*切换 PA AW_DEV_0 为 Off 场景*/
+    aw87xxx_set_profile(AW_DEV_0, aw_profile[1]);
+#endif
    TurnOffDacPower();
}
}
```

5G 平台配置方法

以单 PA 举例：

```
/*以 mt6359 平台为例*/
/*添加驱动接口及场景定义*/
@@ -17,6 +17,13 @@
#include "../codecs/mt6359.h"
#include "../common/mtk-sp-spk-amp.h"
```

```

#ifdef CONFIG_SND_SOC_AW87XXX
extern int aw87xxx_set_profile(int dev_index, char *profile);
static char *aw_profile[] = {"Music", "Off"};
enum aw87xxx_dev_index {
+   AW_DEV_0 = 0,
+};
#endif

/* 添加场景设置函数 */
/*
 * if need additional control for the ext spk amp that is connected
@@ -92,9 +99,25 @@ static int mt6853_mt6359_spk_amp_event(s
switch (event) {
case SND_SOC_DAPM_POST_PMD:
    /* spk amp on control */
#ifdef CONFIG_SND_SOC_AW87XXX
+   ret = aw87xxx_set_profile(AW_DEV_0, aw_profile[0]);
+   if (ret < 0) {
+       pr_err("[Awinic] %s: set profile[%s] failed",
+           __func__, aw_profile[0]);
+       return ret;
+   }
#endif
    break;
case SND_SOC_DAPM_PRE_PMD:
    /* spk amp off control */
#ifdef CONFIG_SND_SOC_AW87XXX
+   ret = aw87xxx_set_profile(AW_DEV_0, aw_profile[1]);
+   if (ret < 0) {
+       pr_err("[Awinic] %s: set profile[%s] failed",
+           __func__, aw_profile[1]);
+       return ret;
+   }
#endif
    break;
default:
    break;

```

2.3 低电量保护功能

2.3.1 dsp 通信接口配置

如果平台带 DSP，请在 aw_dsp.h 中的开启 MTK 平台宏定义：

```
#define AW_MTK_OPEN_DSP_PLATFORM
```

2.4 驱动移植有效性验证

以 aw87560 产品单 PA 举例：

1) I2C 通信正常：


```
[Awinic] aw87xxx_pa_init:driver version: Dv2.4.0.5
[Awinic] [0-0058] aw87xxx_malloc_init: struct aw87xxx devm_kzalloc and init down
[Awinic] [0-0058] aw87xxx_dtsi_parse: parse dev_index=[0]
[Awinic] [0-0058] aw87xxx_dtsi_parse: reset gpio[1160] parse succeed
[Awinic] [0-0058] aw87xxx_device_parse_port_id_dt: rx-port-id: 0xb030
[Awinic] [0-0058] aw87xxx_device_parse_topo_id_dt: rx-topo-id: 0x1000ff01
[Awinic] [0-0058] aw87xxx_dev_hw_pwr_ctrl: hw power on
[Awinic] [0-0058] aw87xxx_dev_get_chipid: read chipid[0x60] succeed
[Awinic] [0-0058] aw_dev_chip_init: product is pid_60 class
```

2) bin 文件加载正常:

```
[Awinic] [0-0058] aw_get_prof_count_v_0_0_0_1: get profile count=[3]
[Awinic] [0-0058] aw_get_vaild_prof_v_0_0_0_1: product=[aw87560]----profile=[Music]
[Awinic] [0-0058] aw_get_vaild_prof_v_0_0_0_1: product=[aw87560]----profile=[Receiver]
[Awinic] [0-0058] aw_set_prof_off_info_v_0_0_0_1: product=[aw87560]----profile=[Off]
[Awinic] [0-0058] aw_get_vaild_prof_v_0_0_0_1: get vaild profile succeed
[Awinic] [0-0058] aw_set_prof_name_list_v_0_0_0_1: index=[0], profile_name=[Music]
[Awinic] [0-0058] aw_set_prof_name_list_v_0_0_0_1: index=[1], profile_name=[Receiver]
[Awinic] [0-0058] aw_set_prof_name_list_v_0_0_0_1: index=[2], profile_name=[Off]
[Awinic] [0-0058] aw_parse_acf_v_0_0_0_1: acf parse success
[Awinic] [0-0058] aw87xxx_init default_prof: init profile name [Off]
[Awinic] [0-0058] aw87xxx_fw_load: acf parse succeed
[Awinic] [0-0059] aw87xxx_fw_load: enter
```

3) Kcontrol 注册正常:

```
[Awinic] [0-0058] aw87xxx_kcontrol_dynamic_create: enter
[Awinic] [0-0058] aw87xxx_kcontrol_dynamic_create: add codec controls[aw87xxx_profile_switch_0,aw87xxx_vmax_get_0,aw87xxx_monitor_switch_0]
[Awinic] [0-0058] aw87xxx_public_kcontrol_create: enter
[Awinic] [0-0058] aw87xxx_public_kcontrol_create: add public codec controls[aw87xxx_spin_switch]
[Awinic] [0-0058] aw87xxx_profile_switch_get: current profile:[Off]
```

4) 切换场景成功 (以设备节点方式为例):

```
kona:/sys/bus/i2c/drivers/aw87xxx_pa/0-0058 # cat profile
Music
Receiver
>Off
kona:/sys/bus/i2c/drivers/aw87xxx_pa/0-0058 # echo Music > profile
[Awinic] [0-0058] aw87xxx_attr_get_profile: current profile:[Off]
[Awinic] [0-0058] aw87xxx_attr_set_profile: set profile [Music]
[Awinic] [0-0058] aw87xxx_update_profile: load profile[Music] enter
[Awinic] [0-0058] aw87xxx_monitor_stop: enter
[Awinic] [0-0058] aw87xxx_power_on: enter
[Awinic] [0-0058] aw87xxx_acf_get_prof_desc_form_name: enter
[Awinic] [0-0058] aw87xxx_acf_get_prof_desc_form_name: get prof desc down
[Awinic] [0-0058] aw87xxx_power_on: get profile[Music] data len [84]
```

5) 播放音乐正常:

```
[Awinic] aw87xxx_set_profile_by_id:aw87xxx, dev_index[0] set profile[Music] by id[1]
[Awinic] [0-0058] aw87xxx_set_profile: set dev_index = 0, profile = Music
[Awinic] [0-0058] aw87xxx_update_profile: load profile[Music] enter
[Awinic] [0-0058] aw87xxx_monitor_stop: enter
[Awinic] [0-0058] aw87xxx_power_on: enter
[Awinic] [0-0058] aw87xxx_acf_get_prof_desc_form_name: enter
[Awinic] [0-0058] aw87xxx_acf_get_prof_desc_form_name: get prof desc down
[Awinic] [0-0058] aw87xxx_power_on: get profile[Music] data len [84]
[Awinic] [0-0058] aw87xxx_dev_hw_pwr_ctrl: hw power on
[Awinic] [0-0058] aw87xxx_dev_reg_update: reg=0x76, val = 0x92
[Awinic] [0-0058] aw87xxx_dev_reg_update: reg=0x7d, val = 0x7a
[Awinic] [0-0058] aw87xxx_dev_reg_update: reg=0x7c, val = 0xc1
```

3. 调试接口

3.1 设备节点

AW87XXX Driver 会创建不同设备节点，路径是 `sys/bus/i2c/drivers/aw87xxx_pa/*-00xx`，其中*为 i2c bus number，xx 为 i2c address。

reg

节点名字	reg
功能描述	用于读写 aw87xxx 的所有寄存器
使用方法	读寄存器值: <code>cat reg</code> 写寄存器值: <code>echo reg_addr reg_data > reg</code> （16 进制操作）
参考例程	<code>cat reg</code> （获取所有可读寄存器上的值） <code>echo 0x01 0x07 > reg</code> （向 0x01 寄存器写入 0x07 的值）

profile

节点名字	profile
功能描述	用于 aw87xxx 注册的 PA 切换场景
使用方法	查看当前场景和可切换的场景: <code>cat profile</code> 设置场景: <code>echo 场景名 > profile</code>
参考例程	<code>cat profile</code> （查看当前场景和可切换的场景） <code>echo "Music" > profile</code> （加载 Music 场景） <code>echo "Off" > profile</code> （加载 Off 场景（PA 下电））

hwen

节点名字	hwen
功能描述	用于控制 aw87xxx 的硬件使能
使用方法	获取 aw87xxx 硬件状态: <code>cat hwen</code> aw87xxx 硬件使能: <code>echo 1 > hwen</code> aw87xxx 硬件关闭: <code>echo 0 > hwen</code>

esd_enable

节点名字	esd_enable
功能描述	用于控制 esd 检测使能
使用方法	读: <code>cat esd_enable</code> 写: <code>echo is_enable > esd_enable</code> （字符串）

参考例程	cat esd_enable echo true > esd_enable echo false > esd_enable	(获取 esd 功能状态) (打开 esd 检测功能) (关闭 esd 检测功能)
------	---	---

vmax

节点名字	vmax
功能描述	用于向 dsp 设置 vmax 和获取 dsp 当前的 vmax 值
使用方法	获取当前 vmax 的值: cat vmax 发送计算后的 vmax 值: echo N > vmax
参考例程	echo 0xffff95f7e > vmax (将 0xffff95f7e 值发送给 vmax)

vbat

节点名字	vbat
功能描述	用于调试设置电量和获取电量值
使用方法	获取实时电量值: cat vbat 设置 vbat 电量值: echo capacity > vbat 取消之前输入的调试电量: echo 0 > vbat
参考例程	echo 50 > vbat (设置当前电量为 50%)

monitor

节点名字	monitor
功能描述	用于控制低电量保护功能
使用方法	获取当前电量的保护状态: cat monitor aw87xxx 关闭自动保护: echo 0 > monitor aw87xxx 开启自动保护: echo 1 > monitor

monitor_count

节点名字	monitor_count
功能描述	设置 vmax 的 monitor 间隔次数
使用方法	获取系统当前电量获取次数: cat monitor_count 设置获取电量求均次数为 5: echo 5 > monitor_count

monitor_time

节点名字	monitor_time
功能描述	用于设置获取电量的时间间隔
使用方法	获取系统当前获取电量的时间间隔: cat monitor_time

设置 N (ms) 时间间隔后获取一次电量: echo N > monitor_time

rx

节点名字	rx
功能描述	用于设置和获取 dsp rx 模块的状态
使用方法	获取 dsp 的 rx 模块的状态: cat rx 设置 dsp 的 rx 模块使能: echo 1 > rx 设置 dsp 的 rx 模块不使能: echo 0 > rx

monitor_update

节点名字	monitor_update
功能描述	用于更新 monitor 参数配置
使用方法	设置更新 monitor 参数配置: echo 1 > monitor_update

4. 附录

4.1 常见问题

4.1.1 移植 awinic 算法后出现白噪

移植 awinic 算法后, PA 工作时若出现规律播放白噪的情况, 时序如下: 播放音源 10s-->播放 3s 白噪-->播放音源 10s-->播放 3s 白噪-->.....

以上现象说明 awinic 算法认证失败, 请使能驱动中对应功能重新编译。

```
diff --git a/aw87xxx_device.h b/aw87xxx_device.h
index 6234a73..ad62b2c 100644
--- a/aw87xxx_device.h
+++ b/aw87xxx_device.h
@@ -64,7 +64,7 @@
/*****
struct aw_device;

-/*#define AW_ALGO_AUTH_DSP*/
+#define AW_ALGO_AUTH_DSP
extern int g_algo_auth_st;

enum AW_ALGO_AUTH_MODE {
```